

Spotify Regression problem

February 24, 2025

0.1 CS985 Spotify Regression: Predicting Song Popularity

Team 101

Team Members: Titirat Seeharach (Student Number: 202580746) Umaima Hussain (Student Number: 202553804) Surya Subramanian (Student Number: 202553846) Calum Murphy (Student Number: 202488764) Zen Gawai (Student Number: 202566417)

0.1.1 Introduction

The goal of this project is to build and train a machine learning model to predict the popularity of a song based on various attributes such as song title, artist, genre, year, BPM (beats per minute), energy, danceability, and more. The dataset provided includes a training set and a test set, each containing 15 and 14 columns, respectively. The target variable is the popularity of the song, represented as an integer value. The performance of the model is evaluated using the Root Mean Square Error (RMSE) metric.

This notebook outlines the steps taken to preprocess the data, select relevant features, and train three different models: Linear Regression, Support Vector Regression (SVR), and Random Forest Regression. The final predictions are saved for submission to Kaggle.

0.1.2 1. Data Preprocessing

0.1.3 1.1 Importing Libraries

The necessary libraries for data manipulation, visualization, and machine learning are imported.

```
[2]: import pandas as pd
import numpy as np
from sklearn.preprocessing import OneHotEncoder, StandardScaler
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression
from sklearn.svm import SVR
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.metrics import mean_squared_error, r2_score
```

0.1.4 1.2 Loading the Data

The training and test datasets are loaded using Pandas.

```
[5]: training_data = pd.read_csv('SpotifyRegression/CS98XRegressionTrain.csv')
      ↪#Ensure file path is correct
test_data = pd.read_csv('SpotifyRegression/CS98XRegressionTest.csv')

print(f'The training data has {training_data.shape[0]} rows and {training_data.
      ↪shape[1]} columns')
print(f'The test data has {test_data.shape[0]} rows and {test_data.shape[1]}
      ↪columns')
```

The training data has 453 rows and 15 columns

The test data has 114 rows and 14 columns

0.1.5 1.3 Handling Missing Values

The dataset is checked for missing values, and any rows with NaN values are dropped.

```
[8]: training_data.isna().values.any() # Check for NaN values
training_data.dropna(inplace=True) # Drop NaN values
print(f'The cleaned training data now has {training_data.shape[0]} rows and
      ↪{training_data.shape[1]} columns')
```

The cleaned training data now has 438 rows and 15 columns

0.1.6 1.4 Exploratory Data Analysis (EDA)

The dataset is explored to understand the distribution of the target variable (popularity) and identify potential outliers.

```
[11]: # Inspect songs with highest popularity
print(training_data[['title', 'artist', 'top genre', 'year', 'pop']].
      ↪sort_values('pop', ascending=False)[:5]) # Top 5 popular songs
```

	title	artist	\
294	Bohemian Rhapsody - 2011 Mix	Queen	
234	The Scientist	Coldplay	
263	Africa	TOTO	
337	Here Comes The Sun - Remastered 2009	The Beatles	
162	Another One Bites The Dust - Remastered 2011	Queen	

	top genre	year	pop
294	glam rock	1975	84
234	permanent wave	2002	83
263	album rock	1982	83
337	british invasion	1969	82
162	glam rock	1980	82

0.2 2. Feature Selection

0.2.1 2.1 Correlation Analysis

A correlation matrix is computed to identify features that are strongly correlated with the target variable (popularity).

```
[14]: training_data.rename(columns={'pop': 'popularity'}, inplace=True) #Rename_
      ↪column as when w
test_data.rename(columns={'pop': 'popularity'}, inplace=True)
corr_matrix = training_data.corr(numeric_only=True)
corr_matrix['popularity'].sort_values(ascending=False)
```

```
[14]: popularity    1.000000
dur              0.321028
dB              0.312952
nrgy            0.274006
dnce            0.256099
spch            0.130346
Id              0.072073
bpm             0.042695
year            0.018926
live           -0.025493
val            -0.040035
acous          -0.443763
Name: popularity, dtype: float64
```

0.2.2 2.2 Encoding Categorical Data

Categorical features such as top genre and artist are encoded using One-Hot Encoding.

```
[17]: test_data.fillna(0, inplace=True) #OneHotEncoding removes NaN values which we_
      ↪don't want for the test data.

#Function that encodes categorical data into binary attributes

encoder = OneHotEncoder()

def encode_data(data):
    data_encoded, data_labels = data.factorize()
    data_1hot = encoder.fit_transform(data_encoded.reshape(-1,1))
    enc_data = pd.DataFrame(data_1hot.toarray())
    enc_data.columns = data_labels
    enc_data.index = data.index
    return enc_data
```

```
[19]: # BEGIN ENCODING GENRE VALUES FOR TRAINING DATA

genres = training_data['top genre']
```

```
genres_encoded = encode_data(genres)
training_data = training_data.join(genres_encoded)
```

```
[21]: # BEGIN ENCODING GENRE VALUES FOR TESTING DATA
```

```
genres_test = test_data['top genre']
genres_test_encoded = encode_data(genres_test)
test_data = test_data.join(genres_test_encoded)
```

```
[23]: # Begin encoding artist values fpr training data
```

```
artists = training_data['artist']
artists_encoded = encode_data(artists)
training_data = training_data.join(artists_encoded)
```

New correlation matrix

```
[26]: new_corr_matrix = training_data.corr(numeric_only=True)
new_corr_matrix["popularity"].sort_values(ascending=False)
```

```
[26]: popularity          1.000000
dur                    0.321028
dB                    0.312952
nrgy                  0.274006
dnce                  0.256099
...
Frankie Vaughan      -0.165044
adult standards      -0.218089
brill building pop   -0.243413
deep adult standards -0.245574
acous                -0.443763
Name: popularity, Length: 429, dtype: float64
```

We can see that some genres have a fairly strong negative correlation with popularity. We will include these features into our model. No artists seem to have a strong correlation so we will not include these in our model and will not encode them for the test data.

0.3 3. Model Training and Evaluation

0.3.1 3.1 Linear Regression

A simple Linear Regression model is trained using the most correlated features.

```
[30]: from sklearn.model_selection import train_test_split
from sklearn.metrics import root_mean_squared_error
```

```
[32]: # Select features and target variable
```

```
features = ['dur', 'dB', 'nrgy', 'acous', 'dnce', 'deep adult standards', 'brill_
↳ building pop', 'adult standards']
X_train = training_data[features]
```

```

y_train = training_data['popularity']

X_test = test_data[features]

#To evaluate performance on training data before kaggle submissions. 80/20
↪split.
X_train_dummy, X_val, y_train_dummy, y_val = train_test_split(
    X_train, y_train, test_size=0.2, random_state=42)

#Train Linear Regression model
lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train)
popularity_predictions = lin_reg.predict(X_test)

#Validation set before Kaggle submission
lin_reg_val = LinearRegression()
lin_reg_val.fit(X_train_dummy, y_train_dummy)
dummy_predictions = lin_reg_val.predict(X_val)

# Save predictions
test_data['pop'] = popularity_predictions
prediction_df = test_data[['Id', 'pop']]
prediction_df.set_index('Id', inplace=True)
prediction_df.to_csv("linear_regression_predictions.csv", index=True)

```

```

[34]: print(f'RMSE of Linear Regression on training set is
        ↪{round(root_mean_squared_error(y_val, dummy_predictions),5)}')

```

RMSE of Linear Regression on training set is 9.21102

Linear regression model yields a rmse score of 8.76528.

0.3.2 3.2 Support Vector Regression (SVR)

An SVR model is trained using a pipeline that includes feature scaling.

```

[38]: # Import make_pipeline
from sklearn.pipeline import make_pipeline

# Create a pipeline with StandardScaler and SVR
model = make_pipeline(StandardScaler(), SVR(kernel='rbf', C=1.0, epsilon=0.1))
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

#Validation predictions before kaggle submission.
dummy_model = make_pipeline(StandardScaler(), SVR(kernel='rbf', C=1.0,
        ↪epsilon=0.1))
dummy_model.fit(X_train_dummy, y_train_dummy)

```

```
dummy_svm_pred = model.predict(X_val)

# Save predictions
submission = pd.DataFrame({'Id': test_data['Id'], 'pop': y_pred})
submission.to_csv('svm_submission_final_v1.csv', index=False)
```

```
[40]: print(f'RMSE of Support Vector Regression on training set is_
      ↪{round(root_mean_squared_error(y_val, dummy_svm_pred),5)}')
```

RMSE of Support Vector Regression on training set is 8.60878

Support vector machine model yields a rmse score of 8.63273 when submitted on kaggle.

0.3.3 3.3 Random Forest Regression

A Random Forest model is trained with hyperparameter tuning using RandomizedSearchCV.

```
[44]: # Define hyperparameter grid
param_grid = {
    'n_estimators': [100, 200, 300, 500],
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': [None, 'sqrt', 'log2']
}
```

```
[46]: # Initialize RandomizedSearchCV
rf = RandomForestRegressor(random_state=42)
rf_random = RandomizedSearchCV(
    estimator=rf, param_distributions=param_grid,
    n_iter=20, cv=5, scoring='neg_mean_squared_error',
    verbose=2, random_state=42, n_jobs=-1
)
```

```
[48]: # Fit the model
rf_random.fit(X_train, y_train)
```

Fitting 5 folds for each of 20 candidates, totalling 100 fits

```
[48]: RandomizedSearchCV(cv=5, estimator=RandomForestRegressor(random_state=42),
    n_iter=20, n_jobs=-1,
    param_distributions={'max_depth': [None, 10, 20, 30],
        'max_features': [None, 'sqrt', 'log2'],
        'min_samples_leaf': [1, 2, 4],
        'min_samples_split': [2, 5, 10],
        'n_estimators': [100, 200, 300, 500]},
    random_state=42, scoring='neg_mean_squared_error',
    verbose=2)
```

```
[49]: # Retrieve best parameters and train final model
best_params = rf_random.best_params_
best_model = RandomForestRegressor(**best_params, random_state=42)
best_model.fit(X_train, y_train)

[49]: RandomForestRegressor(max_depth=30, max_features='sqrt', min_samples_leaf=2,
                           min_samples_split=10, n_estimators=300, random_state=42)

[50]: # Predict on test data
y_pred = best_model.predict(X_test)

[51]: # Save predictions
predictions = pd.DataFrame({'Id': test_data['Id'], 'pop': y_pred})
predictions.to_csv('RF_final_1.csv', index=False)

[52]: #Validation predictions before kaggle submission.

validation_rf = RandomForestRegressor(**best_params, random_state=42)
validation_rf.fit(X_train_dummy, y_train_dummy)
rf_dummy_pred = best_model.predict(X_val)

[53]: print(f'RMSE of Random Forest on training set is_
↳{round(root_mean_squared_error(y_val, rf_dummy_pred),5)}')
```

RMSE of Random Forest on training set is 6.34045

Random Forest model achieved a rmse score of 7.84698 on kaggle submission. This is significantly more than the performance on the training data which suggest possible overfitting.

0.3.4 3.4 Ridge Regression

A Ridge Regression with Polynomial Features to improve performance.

```
[56]: from sklearn.preprocessing import PolynomialFeatures

from sklearn.linear_model import Ridge

# Standardize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_train)

# Use Polynomial Features to improve model performance
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X_scaled)

# Train Ridge Regression (prevents overfitting)
model = Ridge(alpha=1.0)
model.fit(X_poly, y_train)
```

```

# Load the test dataset

# Prepare test data features
X_test = test_data[['dur','dB','nrgy','acous','dnce','deep adult_
↳standards','brill building pop','adult standards']]
X_test_scaled = scaler.transform(X_test) # Use the same scaler as training data
X_test_poly = poly.transform(X_test_scaled)

# Predict on test data
test_data['pop'] = model.predict(X_test_poly)

# Retain only 'id' and 'pop' columns
df_test = test_data[['Id', 'pop']]

# Save predictions
df_test.to_csv("test_predictions_final.csv", index=False)

print("Predictions saved to 'test_predictions.csv'!")

```

Predictions saved to 'test_predictions.csv'!

```

[57]: #DUMMY VALIDATION
X_scaled_dummy = scaler.fit_transform(X_train_dummy)
X_poly_dummy = poly.fit_transform(X_scaled_dummy)

model_dummy = Ridge(alpha=1.0)
model_dummy.fit(X_poly_dummy, y_train_dummy)

X_test_scaled_dummy = scaler.transform(X_val) # Use the same scaler as_
↳training data
X_test_poly_dummy = poly.transform(X_test_scaled_dummy)

ridge_dummy_predictions = model.predict(X_test_poly_dummy)

```

```

[59]: print(f'The RMSE of Ridge Regression on training set is_
↳{round(root_mean_squared_error(y_val, ridge_dummy_predictions),4)}')

```

The RMSE of Ridge Regression on training set is 7.7291

Ridge Regression achieved a score of 7.73066 on kaggle submission. This is the lowest score out of all our models and hence the best one.

0.3.5 4. Justification for Model Selection:

In predicting song popularity, a comparison of regression models revealed Ridge Regression as the top performer with an RMSE of 7.7291. Here we have explored why Ridge Regression stands out

and examines other models considered.

0.3.6 Ridge Regression:

0.4 Ridge Regression excels due to several factors:

1. Precision in Predictions: It offers the lowest RMSE, indicating high accuracy.
2. Handling Multicollinearity: The L2 regularization effectively manages correlated features common in music datasets.
3. Interpretability: Despite its sophistication, it provides clear insights into feature impacts.
4. Overfitting Prevention: Regularization ensures robust performance on unseen data.

Alternative Models: Random Forest Regression (RMSE: 7.84698) • Competitive Performance: Close to Ridge Regression but slightly higher RMSE. • Complexity and Interpretability: More complex and less interpretable than Ridge Regression. Support Vector Regression (SVR) (RMSE: 8.63273) • Higher RMSE: Indicates less suitability for this dataset. • Computational Intensity: Resource-intensive with challenging hyperparameter tuning. Linear Regression (RMSE: 8.76528) • Highest RMSE: Highlights the need for more advanced models. • Assumption Limitations: Assumes linear relationships, which may not hold for music popularity.

Therefore, Ridge Regression is the optimal choice due to its: 1. Superior Accuracy 2. Effective Feature Management 3. Balanced Complexity 4. Robust Generalization While other models offer strengths, Ridge Regression provides the best balance of performance and interpretability for predicting song popularity.

0.4.1 5. Conclusion

Four models were trained and evaluated for predicting the popularity of songs. The performance as shown on kaggle are detailed below:

1. Linear Regression achieved an RMSE of 8.76528.
2. Support Vector Regression (SVR) achieved a cross-validation RMSE of 8.63273.
3. Random Forest Regression achieved an RMSE of 7.84698.
4. Ridge Regression achieved an RMSE of 7.7291

0.4.2 5.1 Linear Regression

- Challenges:
 - Struggles with non-linear relationships, multicollinearity, and outliers. -May underfit complex data, leading to poor predictions.
- Solutions:
 - Use feature engineering (e.g., polynomial features) to capture non-linear patterns. -Apply regularization (Ridge/Lasso) to handle multicollinearity. -Remove or transform outliers to improve model stability.

0.4.3 5.2 Support Vector Regression (SVR)

- Challenges:

-Computationally expensive and sensitive to feature scaling. -Requires careful selection of kernel and hyperparameters.

- Solutions:

-Normalize/standardize features before training. -Experiment with kernels (e.g., RBF, linear) and use grid search for hyperparameter tuning. -Reduce dimensionality with PCA or feature selection to improve scalability.

0.4.4 5.3 Random Forest Regression

- Challenges:

-Prone to overfitting with too many trees or deep trees. -Hyperparameter tuning is complex and computationally intensive.

- Solutions:

-Limit tree depth (max_depth) and increase min_samples_split to prevent overfitting. -Use grid search or randomized search for hyperparameter optimization. -Leverage feature importance to select relevant features and reduce training time.

0.4.5 5.4 Ridge Regression

- Challenges:

-Sensitive to feature scaling and requires tuning of the regularization parameter (alpha). -Assumes linear relationships, which may not hold for all data.

- Solutions:

-Standardize features to ensure consistent scaling. -Use cross-validation to find the optimal alpha value. -Add polynomial or interaction terms to capture non-linear relationships.

The Ridge Regression model performed the best, demonstrating the highest predictive power. Future work could explore more advanced models, feature engineering, and hyperparameter optimization to further improve performance.

0.4.6 6. References

Scikit-learn documentation: <https://scikit-learn.org/>

Pandas documentation: <https://pandas.pydata.org/>

Kaggle competition guidelines.